STAT 6910 Advanced R

Homework 1

by

Jürgen Symanzik

Date: October 19, 2013

Due Date: Friday, November 1, 2013, 11:59pm (by e–mail)

UTAH STATE UNIVERSITY

Logan, UT

Fall 2013

Contents

List of Tables

List of Figures

1 General Instructions

You have to work in small groups of exactly two students on this assignment. If you can't find a partner, please send me an e–mail and I will team you up with someone. We are 16 students — so we will have exactly eight groups of two students.

I would suggest that you first work independently on this assignment and then compare your results with your partner. You can submit only one final version!

You have to submit two files via e–mail: The `.Rnw` file that contains the LaTeX and R/Sweave (or R/knitr) code and the resulting `.pdf` file prepared on your side. Your files should be named `LastName1_LastName2_HW01.Rnw` and `LastName1_LastName2_HW01.pdf`.

Your R code has to be fully reproducible! Make sure to reset the random number seed once as specified in part 2 of the question. You cannot include any constants in your LaTeX text. Rather, you have to work with `\Sexpr` to extract numeric values from your R code. I will likely make changes to your R code, e.g., change some of the numerical values, to check whether your results properly get adapted to these new values.

Use this `.Rnw` file as the basis for your answers. Update the title page, e.g., fill in your names and indicate when this homework gets submitted. Moreover, fill in the chunks of R code where the current file states `### Your R code here`. Before you submit your homework, check once more that you follow all recommendations from Google's R Style Guide (see `http://google-styleguide.googlecode.com/svn/trunk/Rguide.xml`).

For general questions related to this homework, please use the corresponding discussion board in Canvas! I will try to reply as quickly as possible. Moreover, if one of you knows an answer, please post it. It is fine to refer to web pages and R commands, but do not provide the exact R command! This will be the task for each individual group.

2 Question 1 (7×10 Points $= 70$ Points)

Recall that $n!$ (read: "n factorial") is defined as follows: $0! = 1$ and $n! = n \cdot (n-1)!$

1. Write six R functions called `myFactorial1` through `myFactorial6` that calculate $n!$ in six different ways. You can assume that n is a valid positive integer or equal to zero (if not, just let R produce its own error message). These six functions should have the following properties:

   (a) `myFactorial1` should be based on the existing `factorial` function.

   ```
   > ### Your R code here
   ```

   The result should be:

   ```
   > myFactorial1(0)
   [1] 1
   > myFactorial1(1)
   [1] 1
   > myFactorial1(5)
   [1] 120
   > myFactorial1(10)
   [1] 3628800
   > myFactorial1(100)
   [1] 9.332622e+157
   ```

   (b) `myFactorial2` should be based on the `prod` function.

   ```
   > ### Your R code here
   ```

   The result should be:

   ```
   > myFactorial2(0)
   [1] 0
   > myFactorial2(1)
   [1] 1
   > myFactorial2(5)
   [1] 120
   > myFactorial2(10)
   [1] 3628800
   > myFactorial2(100)
   [1] 9.332622e+157
   ```

   (c) `myFactorial3` should be based on a for–loop that multiplies the previous result with the current loop index.

   ```
   > ### Your R code here
   ```

   The result should be:

```
> myFactorial3(0)
[1] 1
> myFactorial3(1)
[1] 1
> myFactorial3(5)
[1] 120
> myFactorial3(10)
[1] 3628800
> myFactorial3(100)
[1] 9.332622e+157
```

(d) `myFactorial4` should be based on a while–loop that checks whether n has been surpassed. If not, the previous result should be multiplied with the current counter.

```
> ### Your R code here
```

The result should be:

```
> myFactorial4(0)
[1] 1
> myFactorial4(1)
[1] 1
> myFactorial4(5)
[1] 120
> myFactorial4(10)
[1] 3628800
> myFactorial4(100)
[1] 9.332622e+157
```

(e) `myFactorial5` should be done in a recursive way, using the fact that myFactorial5$(n) = n \cdot$ myFactorial5$(n-1)$.

```
> ### Your R code here
```

The result should be:

```
> myFactorial5(0)
[1] 1
> myFactorial5(1)
[1] 1
> myFactorial5(5)
[1] 120
> myFactorial5(10)
[1] 3628800
> myFactorial5(100)
```

```
[1] 9.332622e+157
```

(f) `myFactorial6` should be similar to `myFactorial5`, but now write the recursion as $\text{myFactorial6}(n) = \text{myFactorial6}(n-1) \cdot n$.

```
> ### Your R code here
```

The result should be:

```
> myFactorial6(0)
[1] 1
> myFactorial6(1)
[1] 1
> myFactorial6(5)
[1] 120
> myFactorial6(10)
[1] 3628800
> myFactorial6(100)
[1] 9.332622e+157
```

2. We want to determine which of these six factorial functions is fastest. To do so, we want to run each of these functions l consecutive times and repeat this k times. So, for each of the six functions, we will have k blocks of l consecutive times this function is run.

   A general principle in statistics is a fully randomized experiment. We should **not** simply run `myFactorial1` $k * l$ times, followed by `myFactorial2` $k * l$ times, and so on. There might be other things happening on your computer that slow down your computer at a particular time (e.g., downloading of software upgrades in the background, virus checking, web browsing, or garbage collection). Therefore, we need to randomize the order. However, it will be fine if it happens by chance that one of the functions is run l consecutive times and then immediately is run again l consecutive times. There is no need that in the first six (times l) runs all of the six different functions are run. The examples below should clarify this part.

   Write a function called `CreateSampleOrder` that creates such a random order. The first argument should be called `options` (with no default value) and the second argument should be called `blocks` (with a default value of 1). We will use `options = 6` in this assignment, but someone else might want to compare more or fewer functions. Therefore, do not assign any default value to options.

   To make your results fully reproducible, you must reset the random number generator. Use the combination of the days of your birthday as a seed, e.g., if your birthdays are October 18 and December 31, use the following:

```
> set.seed(1831) # or use set.seed(3118)

> ### Your R code here
```

   Here are some possible results:

```
> CreateSampleOrder(6)
```

```
[1] 6 2 1 5 4 3

> CreateSampleOrder(6)

[1] 2 3 1 6 5 4

> CreateSampleOrder(6, 2)

 [1] 2 6 5 6 1 1 5 4 2 4 3 3

> CreateSampleOrder(6, 2)

 [1] 2 1 6 5 3 1 5 4 3 6 2 4

> CreateSampleOrder(6, 5)

 [1] 5 6 2 1 4 4 5 4 3 4 2 3 1 2 3 6 3 5 1 6 2 2 1 6 5 3 4 1 5 6

> mySampleOrder = CreateSampleOrder(options = 6, blocks = 5)
> mySampleOrder

 [1] 5 1 4 2 5 4 5 6 4 2 6 2 1 3 4 3 3 1 6 5 2 1 1 5 6 6 3 3 2 4
```

3. Now, write a function called `DetermineRunTimes`. The arguments for this function should be `sampleOrder`, `l` (with a default value of 10) that indicates how many consecutive times each of the six factorial functions will be run, and `n` that determines for which value the factorial is calculated (and has a default value of 100). We want to measure the total time for each of the k blocks of the l consecutive runs of the factorial function (as specified by `sampleOrder`). The result of the `DetermineRunTimes` function should be a matrix with six columns (representing the six factorial functions) and k rows that contain the times for the l consecutive runs for each of the k runs of the corresponding factorial function. Name the columns of the matrix according to the names of the six factorial functions.

   Hint: You need some additional help vector(s) that count how many times each of the six functions has been tested so far so that you fill in the right row (and column) of the matrix. Also note that we only are interested in the `user` component of `system.time`

```
> ### Your R code here
```

   Initially, test your code with:

```
> allRunTimes = DetermineRunTimes(sampleOrder = mySampleOrder,
+                                 l = 1000, n = 10)
> allRunTimes

     myFactorial1 myFactorial2 myFactorial3 myFactorial4 myFactorial5
[1,]         0.00         0.00         0.00         0.03         0.03
[2,]         0.00         0.02         0.02         0.03         0.03
[3,]         0.01         0.02         0.02         0.04         0.03
[4,]         0.00         0.00         0.02         0.03         0.03
[5,]         0.00         0.00         0.02         0.03         0.03
```

```
     myFactorial6
[1,]         0.03
[2,]         0.03
[3,]         0.03
[4,]         0.05
[5,]         0.03
```

Note that several of the run times will likely be identical within each of the six functions (and this will cause problems in the t–test in part 5. later on as the variance will be equal to zero). You gradually have to increase `n` to obtain different run times for each of the six functions.

Finally, once you are sure that everything in your code is correct, run the following:

```
> mySampleOrderFinal = CreateSampleOrder(options = 6, blocks = 100)
> length(mySampleOrderFinal)

[1] 600

> head(mySampleOrderFinal, n = 100)

  [1] 2 1 1 6 2 1 1 1 4 3 5 1 5 4 3 5 1 6 6 3 5 3 4 4 4 3 2 2 2 3 4 1 3 6 4 2 1
 [38] 5 2 1 6 2 1 1 4 1 1 5 6 6 4 5 5 1 3 4 6 3 2 2 1 5 4 2 5 6 2 3 5 2 5 3 4 3
 [75] 1 5 2 3 6 1 3 6 5 1 1 1 2 6 3 3 1 5 2 2 5 1 6 4 1 5

> ### Create a meaningful plot here that shows that each of the
> ### numbers 1 through 6 indeed shows up 100 times in mySampleOrderFinal.
>
> ### Your R code here
>
> allRunTimes = DetermineRunTimes(sampleOrder = mySampleOrderFinal,
+                                 l = 10000, n = 100)
> dim(allRunTimes)

[1] 100   6

> allRunTimes[1:10, ]

      myFactorial1 myFactorial2 myFactorial3 myFactorial4 myFactorial5
 [1,]         0.03         0.02         0.98         2.41         3.00
 [2,]         0.01         0.04         0.98         2.57         3.08
 [3,]         0.03         0.03         0.98         2.56         3.01
 [4,]         0.03         0.03         1.00         2.48         3.28
 [5,]         0.04         0.03         0.99         2.47         2.95
 [6,]         0.03         0.04         0.98         2.51         2.95
 [7,]         0.01         0.03         0.93         2.45         3.05
 [8,]         0.03         0.03         1.01         2.67         3.13
 [9,]         0.03         0.03         0.94         2.56         2.95
[10,]         0.02         0.03         0.97         2.40         2.97
      myFactorial6
```

```
 [1,]         3.04
 [2,]         3.14
 [3,]         3.13
 [4,]         2.94
 [5,]         3.00
 [6,]         3.03
 [7,]         3.16
 [8,]         2.99
 [9,]         2.94
[10,]         3.03
```

Initially, while still debugging your R code and working on the formatting of your LaTeX document, you should comment out all of these lines via %. When you are finally ready to run this, you need to transfer this back into a true R code chunk by removing the % symbols. **Enjoy your dinner or keep this running overnight!!!** ☺

4. Create side–by–side boxplots of the run times for the six different factorial functions. What does this graphic tell you? Are the data symmetric or skewed? If so, to which side? Are there any outliers? If so, explicitly mention them. It is not sufficient to write "as we can clearly see in the figure ...". It is your responsibility as the author of a document to summarize what can be seen in a figure!

```
> ### Your R code here
```

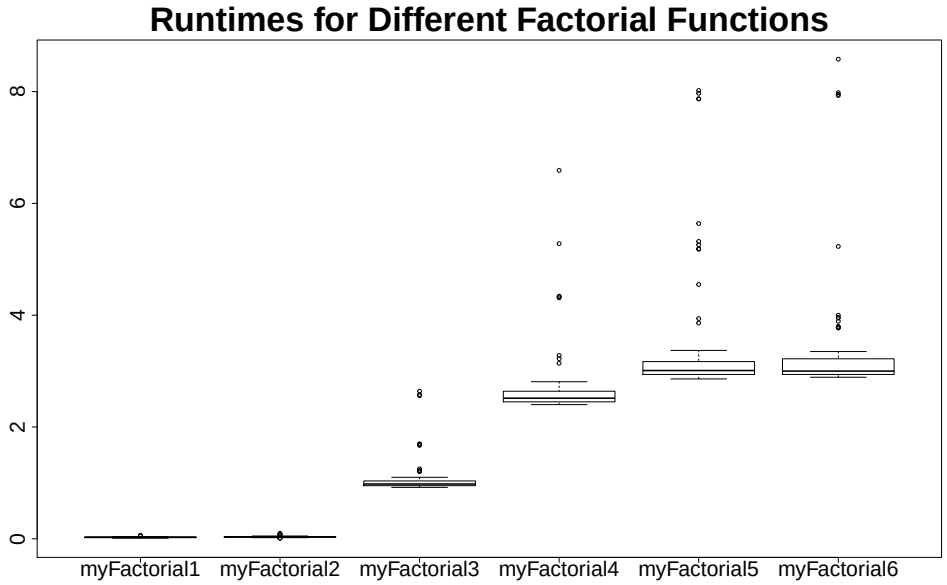

Figure 1: Boxplots for the run times of the six different factorial functions.

5. We are statisticians, aren't we? So, conduct all two–sided pairwise t–tests with a Bonferroni adjustment. You need to figure out the proper R command(s) yourself! Do **not** pool the SD! You need to understand the help page for this specific function — and then apply this to your `allRunTimes`.

```
> ### Your R code here

        Pairwise comparisons using t tests with non-pooled SD

data:  allRunTimesVector and groupingVector

             myFactorial1 myFactorial2 myFactorial3 myFactorial4 myFactorial5
myFactorial2 7.1e-07      -            -            -            -
myFactorial3 < 2e-16      < 2e-16      -            -            -
myFactorial4 < 2e-16      < 2e-16      < 2e-16      -            -
myFactorial5 < 2e-16      < 2e-16      < 2e-16      9.9e-07      -
myFactorial6 < 2e-16      < 2e-16      < 2e-16      2.7e-06      1

P value adjustment method: bonferroni
```

6. Create a table that lists the mean, median, and standard deviation (SD) for the six different factorial functions. Round to three decimal digits. The function `sprintf` might be helpful here. Try some of the examples on the help page to better understand this function!

Table 1: Mean, median, and SD for the run times of the six different factorial functions.

	Mean	Median	SD
`MyFactorial1`	0.026	0.030	0.010
`MyFactorial2`	0.036	0.030	0.013
`MyFactorial3`	1.070	0.980	0.309
`MyFactorial4`	2.668	2.515	0.588
`MyFactorial5`	3.368	3.010	1.083
`MyFactorial6`	3.320	3.000	1.040

7. Provide an overall summary and discuss your results. Which function would you recommend to use for the factorial?

   Note: In any document, you **must** reference each figure and each table in the main text and you **must** also provide a caption for each. Statements such as "the table above shows ..." are not acceptable. Instead, you must refer to them by number, e.g., "Figure 1 shows ..." or "Table 1 provides ...". Capitalize *Figure* and *Table* if you refer to a specific figure or table. However, if you continue to refer to the same figure or table, then use lowercase, e.g., "this figure also shows ...".